

NOMBRE: _____

Vamos a realizar un programa para una empresa de transporte donde, entre otras cosas, se gestionan paradas y trayectos de autobús.

Usando el framework Pedrosa final con los componentes añadidos vamos a crear una pequeña aplicación para gestionarlo. (asegurarse que no cargue la base de datos ni ACL para evitar posibles errores)

Se entregará un fichero comprimido con todo el contenido del proyecto. El archivo se llamará "apellidosnombre-2evaluacion.rar".

Se deben usar, siempre que haya, los componentes del framework existentes.

1.- (0.75 puntos) Crear la clase Lista

Crear la clase **Lista** en la carpeta **aplicacion/varios** (se debe poder cargar automáticamente). Esta clase contendrá métodos estáticos que nos darán listas de elementos.

Crear en esta clase el método estático **listaTrayectos** con los parámetros **\$completo** (booleano, por defecto false) y **\$cod_trayecto** (entero, por defecto null). Internamente tendremos un array con todos los tipos de trayectos:

```
[3 => ["nombre" => "urbano corto", "num_paradas" => 5, "precio" => 1.50], 5 => ["nombre" => "urbano largo", "num_paradas" => 10, "precio" => 2.00], 8 => ["nombre" => "interurbano", "num_paradas" => 15, "precio" => 3.50]]
```

Si **\$completo** vale true, se devolverá para cada tipo todos los datos (nombre/num_paradas/precio), si vale false solo se devolverá el nombre. Si no se indica **\$cod_trayecto** devolverá un array con los trayectos posibles (cada posición código => datos –según \$completo). Si se indica **\$cod_trayecto**, devolverá los datos del trayecto (según valor de \$completo) si existe el código, o false si no existe.

2.- (1.75 puntos) Controlar las paradas posibles (trayecto – estación – población)

Para ello crear el modelo **Parada**. El modelo tendrá los siguientes campos:

- **cod_trayecto** (entero, obligatorio, debe ser uno de los códigos de tipos de trayectos del método listaTrayectos de la clase Lista; por defecto, el código de la posición media en la lista – no quiero que pongas ningún valor directamente sino que lo obtengas del array de los códigos de tipos posibles).
- **nombreTrayecto** (cadena 30, nombre del tipo de trayecto obtenido a partir de cod_trayecto).
- **estacion** (cadena, 30 caracteres, obligatorio, por defecto '-').
- **poblacion** (cadena 30, obligatorio, por defecto se obtendrá usando la función de error).
- **es_origen** (entero, 0 o 1, indica si la parada es de origen; por defecto 0).

Los métodos del modelo serán **setDescripcion** y **afterCreate**. Las descripciones de los campos deben empezar por "Para-". Deben asignarse inicialmente los valores por defecto cuando se cree un objeto de este modelo.

3.- (1.5 puntos) Controlador ParadasTray (controlador por defecto)

En el controlador tendremos el array **ParadasTray**. En este array se mantendrán todas las paradas que tengamos. Hay que garantizar que al insertar una parada esté disponible para las siguientes llamadas (páginas).

Inicialmente este array tendrá dos Paradas (con los datos que queráis, distintos de los valores por defecto, distintos de una parada a otra y con el campo **nombreTrayecto** sin asignar) y se debe garantizar que todos los campos estén bien rellenos. La posición en este array será un entero que se corresponde con el **cod_trayecto** y que usaremos para referirnos a la parada correspondiente.

Se definirá la variable de aplicación **N_Paradas** que contendrá el número de paradas que actualmente se tiene (garantizar que tiene un valor correcto).

Además, tendremos la variable del controlador **N_ParadasOrigen** que contendrá el número de paradas que son de origen (es_origen = 1).

4.- (0.75 puntos) Acciones login y logout

En el controlador ParadasTray, se tendrán las acciones **login** y **logout** que permitirán respectivamente registrar/quitar registro de un usuario.

Se permitirá el registro si tenemos 1 o más paradas registradas en el array. Si se puede, se registrará un usuario con nombre/nick tu nombre y permisos 2, 4 y 6. Se podrá quitar el registro si hay un usuario registrado y tenemos 2 o más paradas (sea cual sea el trayecto). Tanto en el login como en el logout, se mostrará una página de error si no puede hacerse.

5.- (0.75 puntos) Modificar la plantilla

Modificar la plantilla para que aparezca una nueva barra sobre el pie donde se mostrarán "**Paradas: N_Paradas**", "**Paradas origen: N_ParadasOrigen**", el nombre del usuario validado (o usuario no validado) y enlaces para conectar/desconectar (aparecerán según haya o no usuario validado). La barra tendrá el fondo naranja y el color de letra blanco. Los enlaces deben llevar a la operación correspondiente.

6.- (1.5 puntos) Acción principal (acción por defecto del controlador)

Se define la acción **Ver** que será la acción por defecto del controlador ParadasTray. Mostrará:

- Enlace a nueva: con imagen /imagenes/16x16/nuevo.png y texto NUEVA PARADA.
- Un formulario con un combo con las estaciones posibles (se muestran estaciones distintas de las paradas existentes) y un botón ver. Al enviar el formulario se mostrarán todas las paradas definidas para la estación indicada en forma de páginas (pueden aparecer una detrás de otra) usando una vista y una vista parcial por cada parada. Para cada parada tendremos un enlace a la acción descargar (para descargar los datos de esa parada). Se debe mantener la estación seleccionada en el combo.
- Los datos de las paradas se pasarán a la vista usando modelos.

7.- (2 puntos) Acción Nueva

La acción **Nueva** mostrará un formulario para introducir una nueva parada. Con las siguientes características:

- Se recogerán el cod_trayecto, la estación y la población.
- Los campos **cod_trayecto** se recogerá mediante un componente de modelo con un select combo, y se mostrará su etiqueta y su error. El campo **estacion** se recogerá con un componente campo (no modelo), mostrando etiqueta y error. El campo **poblacion** se recogerá con un componente etiqueta-modelo, campo texto y error.
- Para indicar el cod_trayecto se usará un select con los tipos de trayectos indicados en la clase Lista.
- Además de las condiciones indicadas en el modelo, se comprobará que la estación tenga al menos 5 caracteres (error en estación).
- Si al verificar los datos hay algún error se mostrará de nuevo el formulario con los datos introducidos y los errores.
- Si todo es correcto se añadirá la parada al array ParadasTray y se garantizará que sea accesible en peticiones posteriores.

8.- (1 puntos) Acción descargar

La acción **descargar** permitirá devolver un fichero XML (mime application/xml) con los datos de la parada indicada. Los datos se recogerán en la acción y se pasarán a una vista que será la encargada de crear la descarga. El formato será:

```
<para>
  <cod_trayecto>...</cod_trayecto>
  <estacion>...</estacion>
  <poblacion>...</poblacion>
  ... (todos los campos)
</para>
```

Esta opción dará un error si no hay usuario o no tiene el permiso 6.

Apartado	Descripción	PB	Nota
Ejercicio 1 (Clase Lista) (0.75 pts)	Clase en aplicacion/variados, carga automática. Método estático listaTrayectos: devuelve solo nombres o datos completos, devuelve todos o uno concreto.	0.75	
Ejercicio 2 (Modelo Parada) (1.75 pts)	Campos: cod_trayecto, nombreTrayecto, estacion, poblacion, es_origen. Métodos: setDescription, afterCreate. Validaciones y valores por defecto.	1.75	
Ejercicio 3 (Controlador ParadasTray) (1.5 pts)	Array ParadasTray con 2 paradas iniciales. Sesión: carga/recogida/almacena. Variable aplicación N_Paradas y variable controlador N_ParadasOrigen.	1.5	
Ejercicio 4 (Login/logout) (0.75 pts)	Login si ≥ 1 parada hoy, registra usuario con permisos 2,4,6. Logout si usuario registrado y ≥ 2 paradas. Error si no se puede.	0.75	
Ejercicio 5 (Plantilla) (0.75 pts)	Barra sobre pie: N_Paradas, N_ParadasOrigen, usuario/login, fondo naranja/texto blanco.	0.75	
Ejercicio 6 (Acción principal) (1.5 pts)	Controlador/acción por defecto. Enlace a nueva, enlace descarga por parada. Combo estaciones, mantiene selección, filtra, vista/subvista, pasa modelos.	1.5	
Ejercicio 7 (Acción nueva) (2 pts)	Formulario (etiq-modelo combo-error, etiq-campo text-error, etiq-modelo text-error). Comprobar estación, añadir error al modelo, validar modelo, mostrar datos si error, añadir al array.	2	
Ejercicio 8 (Descarga) (1 pts)	Validar usuario/permiso 6, generar error, cabeceras XML, codificar XML con todos los campos.	1	
General		-1	
VALORACIÓN			10